

Automatic Detection of Pneumonia using Concatenated Convolutional Neural Network

Ahmad T. Al-Taani (✉ ahmadta@yu.edu.jo)

Yarmouk University

Ishraq T. Al-Dagamseh

Yarmouk University

Research Article

Keywords: Deep learning, Convolutional Neural Network, Chest X-rays, Pneumonia, Pre-activation ResNet, DenseNet169

Posted Date: November 7th, 2022

DOI: <https://doi.org/10.21203/rs.3.rs-2220817/v1>

License: © ⓘ This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Abstract

Pneumonia is a life-threatening disease and early detection can save lives, many automated systems have contributed to the detection of this disease and currently deep learning models have become one of the most widely used models for building these systems. In this study, two deep learning models are combined: DenseNet169 and pre-activation ResNet models and used for automatic detection of pneumonia. DenseNet169 model is an extension of the ResNet model, while the second is a modified version the ResNet model, these models achieved good results in the field of medical imaging. Two methods are used to deal with the problem of unbalanced data: class weight, which enables to control the percentage of data to be used from the original data for each class of data, while the other method is resampling, in which modified images are produced with an equal distribution using data augmentation. The performance of the proposed model is evaluated using a balanced dataset consists of 5856 images. Achieved results were promising compared to several previous studies. The model achieved a precision value of 98%, an area under curve (AUC) based on ROC of 97%, and a loss value of 0.23.

1. Introduction

Pneumonia is a disease that targets the lungs. The cause of the disease, whether it is a virus, bacteria, or fungus [1], seeks to damage one or both lungs. According to a report issued by the world health organization (WHO), it has indicated that pneumonia is one of the most serious life-threatening diseases, especially for children under the age of five, as it kills about 1.2 million children of this category around the world [2]. Other statistics have shown how dangerous and threatening pneumonia can be to a person's life. For example, a study indicated that there are about 4.5 million people infected with pneumonia around the world each year and more than 50,000 people die from this disease in the USA each year [3]. In another study in 2012, it was indicated that there were about 28,952 deaths in the UK that were caused by pneumonia that year. In two statistical studies in 2016, the first concluded that about 880,000 children under the age of two died from this disease [4], while the second statistic in the same year indicated that Indian reports indicated that the highest death rates around the world were due to pneumonia with about 158,176 deaths in that year [5]. According to a report by the WHO, one out of every three deaths is due to pneumonia in India [6]. Figure 1 shows three x-ray images for a normal, bacterial pneumonia, and pneumonia images.

From Fig. 1 we can clarify the difference between the normal chest image in the left, specifically by focusing on the lungs area, which appears in a black color that expresses the presence of gases and their normal exchange. The middle image expresses a lung infected with pneumonia of a bacterial type and the affected area is the one to which the two arrows point which appears as a blurring or gray area. The image on the right shows two lungs infected with the viral type and also appears in a hazy color, the difference between the bacterial and viral types is that the bacterial infects one side of the lungs, while the viral may infect both lungs.

The radiologists focus on identifying white spots and foggy or blurry areas to locate the injury, but the fact that the x-ray images of the chest are in black and white, causes another challenge for the diagnostician to make a proper diagnosis of the disease [7]. This needs experience and accuracy, in addition to the overlap of these symptoms with other diseases such as lung cancer, which appears as opacity in the picture [8]. Therefore, there is a need to build automated systems capable of diagnosing this disease with greater accuracy and less time. The stages of developed algorithms and methods used for diagnosing pneumonia can be divided into three main stages: the oldest stage which started in the nineties used textual analysis to detect the CXR images [9–10]. As for the intermediate stage, it started with the emergence of machine learning algorithms like support vector machine (SVM), k-nearest neighbor (KNN), Naïve bayes (NB), and random forest (RF) [11][12]. The research in the current period i.e., the third stage focused on the use of deep learning models, specifically convolutional neural network (CNN) models, such as VGGNet, AlexNet, and GoogleNet [13], and ResNet models like ResNet-38, ResNet-50, and ResNet-101 [3][14][15]. Deep learning models are mostly used for feature extraction and some focus on identifying regions of interest in the image, i.e., identifying the most important and effective area in diagnosing the disease [8].

In this paper, we proposed a new model consisting of two integrated models of deep learning models for the phase of extracting characteristics from CXR images to automatically diagnose pneumonia, and those models are: pre-activation and ResNet. These models have demonstrated their efficacy and accuracy in medical and clinical settings [16]. ResNet is merged with the pre-trained DensNet169 model, which trained on a large dataset (ImageNet). The other goal of this study is to show the importance of supporting the numerical results to evaluate the performance of the model with what is called the performance curve, which shows how the model learns during the training and validation processes.

The rest of this paper is organized as follows: In the section 2, we present the most prominent previous studies, specifically those that focused on deep learning models and supported their results with a performance curve. Section 3 presents the proposed model in detail. Section 4, presents and discusses the experimental results. Finally, conclusions drawn from this study and directions for future work are presented in section 5.

2. Related Work

Most of the recent studies, specifically those that were used to build systems for automatic detection of pneumonia, focused on the use of deep learning models in general and CNN models, as they have a high ability to extract the smallest details from images. We have highlighted in this section the most related studies that used deep learning algorithms on the same database proposed in [17].

Most of the previous approaches used pre-trained models on large datasets. In [18] three types of pre-trained CNN models are used in the early feature extraction stage, these include AlexNet, VGG-16 and VGG-19. Five algorithms are used while in the classification stage including KNN, decision tree (DT), linear discriminant analysis (LDA), linear regression (LR) and SVM. The best accuracy they reached was

about 99%, but the performance curve showed a higher fluctuation in the validation phase than in the training phase.

In [19], a DenseNet-121 pre-trained model is used as a feature extractor with a deep neural network as a classifier. The best accuracy reached is 98% after conducting three experiments on the model, while the shape of the performance curve was unstable in the first and third experiments and reached a more stable curve in the second experiment.

Another study based on pre-trained models is proposed by Chouhan et al. [1] using five pre-trained CNN models: ALexNet, DenseNet121, InceptionV3, ResNet18, and Google Net. The proposed model achieved an accuracy of 96.39%, and slightly squiggly performance curves for all models.

In [20], a pre-trained model (Xception) is used in the feature extraction step, and an under-sampling technique is used to re-balance the dataset. The proposed model reached a value of 96% for AUC measure, a recall of 99%, a precision of 84%, an F1-score of 0.91, and a validation loss of 0.04. The performance curves started to fluctuate until the 20th epoch, which is where the stability started perfectly.

In [21], five pre-trained CNN models: Xception, Inception-v3, VGG-16, ResNet50, and DenseNet201, are suggested to extract features from chest x-rays (CXR), then extracted features by every model are combined using Dempster–Shafer theory. The performance curve measure is not used for the evaluation of the model; only accuracy, precision, and f1-score metrics are used. The model obtained an accuracy of 97%, a recall of 98%, and an AUC of 99%.

In [22], Islam et al. proposed a model consisted of two pre-trained CNN models: SqueezeNet and Inception-V3, to extract attributes from CXR images. SVM, KNN, and Artificial Neural Network (ANN) are used for classification. The best reported results were when using the ANN, which gave an accuracy and a sensitivity of 98%, and a specificity and a precision of 99%.

Ayan and Ünver [23] compared the performance of the two pre-trained models Vgg16 and Xception. The results showed that the Vgg16 model is outperformed the Xception model with an accuracy of about 87%, while the Xception model achieved an accuracy of 82%. For the performance curves, turbulence and fluctuations are evident in the performance curve with respect to the validation stage of the VGG16 model, while the performance curve seemed less turbulent with respect to the Xception model with a difference in the results of the two curves in the training and the validation phases.

A pre-trained ResNet-50 model is employed by Hussain et al. [24] for classifying injured and non-injured CXRs, data augmentation is also used to increase the number of images in the dataset. The model obtained an accuracy of about 90%, while the performance curve is very turbulent and has little stability in the validation phase.

Singh et al. [25] built a modified model of ResNet, called Quaternion CNN. The proposed model differs from the original ResNet model in which every convolutional layer is replaced by a Quaternion convolutional layer, also every Relu layer is replaced by an exponential linear unit. The proposed model

reached an accuracy of 93%, loss value of 0.25%, and an F1-score of 94%. The shape of the performance curve was turbulent until the fortieth step and then seemed perfect, also the loss scale was stable in shape, and the difference between the training curve and the validation was increasing over time.

Rači et al. [26] applied the traditional CNN model layer by layer where the accuracy of that model reached 90%. The performance curve for the validation stage was highly fluctuating and more pronounced in the loss scale than in the performance curve for the accuracy measure, which, although fluctuated, was more fluid and the difference between the two curves in the training and validation stages is less. In [3] CNN and ResNet models are combined and used for the detection of pneumonia. Also, Jain et al. [27] used six CNN models, and four pre-trained models: VGG16 and VGG19, Inception V3, and ResNet50, for pneumonia detection. The models obtained good results, while the performance curve for all models, was unstable with a clear difference between the performance of the training and validation curve. Table 1 present a summary of the key papers used in this study.

Table 1
Summary of the key papers

Reference	Approach	Limitations	Best Results
[1]	CNN model	Slightly zigzag curves in the performance curve.	Accuracy: 96.39% Recall: 99%
[3]	CNN with Lightened Image on Increased Contrast with ResNet	A low value of accuracy with no consideration of the performance curve.	Accuracy: 78.73%
[19]	Lightweight Deep ANN	Unstable performance curve	Accuracy: 98%
[20]	Xception pre-trained model	Different results for different performance measures.	AUC: 0.96%, recall: 0.99, precision: 0.84% F1-score: 91%
[23]	Vgg16 and Xception models	Oscillating curves of both models.	Accuracy: 87%
[26]	CNN	The performance curve was too squiggly in the validation phase.	Accuracy: 90%
[27]	CNN, VGG16, VGG19, ResNet50, InceptionV3.	The performance curve is unstable for all models.	Accuracy: 92% and 88%,

3. Methodology

The overall architecture of the proposed approach is presented in Fig. 2. It comprises three phases: preprocessing; data augmentation and normalization, feature extraction, and classification.

3.1. Preprocessing

Data augmentation is used to increase the volume of the dataset since deep learning models need large data to give high performance. It is also used to address the problem of overfitting, i.e., the model achieves good results on the training data, while it doesn't perform well on new unseen data [28]. Eight augmentation methods are applied on this study as presented in Table 2. All these operations are performed randomly to produce new images, where more than one operation was applied [29]. Image sizes for the dataset are normalized to 224x224 to be suitable for the DenseNet169 pre-trained model.

Table 2
The proposed data augmentation methods

Augmentation process	Description
Rescale (1/255)	Transforming the image pixels values from numbers between 0 and 255 to numbers between 0 and 1.
Rotation = 40	Rotating images between 1° and 40° degrees.
Width-shift = 0.2	Shifting images horizontally with a percent of 0.2.
Height-shift = 0.2	Shifting images vertically with a percent of 0.2.
Zoom-range = 0.2	Scaling and inverse scaling of images.
Horizontal flip = True	Flipping images horizontally.
Fill-mode = 'nearest'	Filling the nearest of the body shape with points (deciding the border of the image).
Shear-range = 0.2	Shearing of images.

3.2. Feature Extraction

Two CNN models are used in this study including pre-activation ResNet and DenseNet169. The Pre-activation ResNet model is built from scratch, i.e., defined layer by layer, and the DenseNet169 model is applied using the weights used during the training phase on the ImageNet dataset.

3.2.1. Pre-activation ResNet Model

Pre-activation ResNet is a special version of ResNet proposed by He et al. [30]. The ResNet model has two different architectures based on the flow of data. The first architecture follows a main path, while the second architecture takes a shortcut path, i.e., data can travel between layers and bypass the others depending on the type of main path [31].

The first architecture consists of an arrangement of layers: a convolutional layer, a batch normalization layer, and a rectified linear unit (Relu) layer or a bottleneck. The second architecture is based on the same principle with a different arrangement of the layers so that their arrangement in the main path is as

follows: BN layer, a Relu layer, and a convolutional layer (conv layer). Figure 3 shows the two different architectures of the ResNet.

Equation 1 represents the block of the pre-activation ResNet, where the output layer in the pre-activation ResNet from the main is $Xl + 1$ and shortcut paths Xl is the input features, F is the path, $\circ F$ is the activation function, and Wl is the weight.

$$Xl + 1 = Xl + F(\circ F(Xl), Wl) \quad (1)$$

BN layer is based on the principle of estimating the means and standard deviation of the inputs of each layer for each small batch or subset of the training set. Bjorck, *et al.* [32] defined BN as “a normalization method for layer activation in the hidden layers of deep learning models”. BN layer is used to increase training speed and to improve accuracy in deep learning models [33].

Relu is the most widely used type of activation function in deep learning models. It defines the output of the hidden layer inputs and works by converting all numbers less than zero to zero and keeping the other numbers, Fig. 4 represents the Relu layer.

Equation 2 represents how the Relu works by converting each number less than zero to be zero and keeping the other on their value.

$$f(x) = \max(0, x)$$

2

Conv layer is the third layer on the pre-activation ResNet block, it works by moving the filter on the images to extract the features. This process is done by multiplying the value of the image's pixel array with the filter numbers, which produce what is called a feature map [34]. In practice, that sequence is defined iteratively again in the main path as we define one conv layer in the shortcut path. As for the layers before those blocks, the conv layer is defined, followed by the max-pooling layer to reduce the size of feature maps. After defining the blocks, we must define (BN => Relu) [35]. The next layer is the pooling and flatten layer; its function is to reshape the feature maps from a two-dimensional array to a one-dimensional array to be feed into the ANN classifier after merged it with other features from the DenseNet169model. Figure 5 shows the main layers of the proposed pre-activation ResNet model.

3.2.2. Pre-trained DenseNet169 Model

DenseNet169 is a type of densely connected convolutional layer with 169 layers. It is proposed by Huang et al. [36] and considered as one of the most popular CNN models used by researchers for classification and segmentation tasks. DenseNet169 is trained on many large datasets, but the volume of these datasets is less than those used by some researchers like Kundu et al. [37] and El Asnaoui et al. [38]. In this study, we used the DenseNet169 model with the ImageNet datasets for feature extraction and ANN is used for classification. Figure 6 shows a DenseNet model with 5 layers. The pre-trained DenseNet169

model is applied after resizing the image size into 224x224. The features extracted from the two models are combined using concatenation layer [39].

3.3. Classification

The network consists of one hidden layer, 450 neurons, a Relu activation function, and one output layer with one neuron. ANN is used for classification and the sigmoid activation function is also used in this study since it is commonly used for similar tasks [40].

Formula 3 is used for feature extraction using two models based on certain number of samples or specific batch, while Formula 4 is used to compute the value of the sigmoid function [41].

$$Z = \sum_{i=1}^m W_i X_i + \text{Bias}$$

3

$$\sigma(Z) = \frac{1}{1 + e^{-Z}}$$

4

4. Experimental Results

In this section, we present the dataset used in the research, and the most important experiments conducted using performance curves, which show the stability of the model's performance during training and validation.

4.1. Dataset

The dataset proposed by Kermany et al. [17] is used to evaluate the proposed approach. The dataset consists of 5856 CXR images of pneumonia two classes: infected images and not infected images. This dataset is for children within ages between 1 and 5 years and classified by specialists in the “women and children's medical center” at Guangzhou [42]. The dataset is divided into 79% for training, 19% for testing, and 2% for validation. But the dataset is re-split into 75% for training and 25% for validation, and the same number for prediction images. Table 3 shows the distribution of the original dataset in detail.

Table 3
The original distribution of the proposed dataset

	Infected	Normal	Total
Training	3875	1341	5216
Testing	390	234	624
Validation	8	8	16
Total	4273	1583	5856

4.2. Evaluation Metrics

The performance of the proposed model is evaluated using five metrics: accuracy, precision, recall and AUC, and it is the rate between the TP and FP. In general, it is used in binary classification models, and it is used to evaluate models that take into account the same percentage of importance for the classification of both types of samples.

Binary cross entropy is also used as a loss function to measure the difference between the estimated probability and the actual probability [43]. These metrics are formulated using Formulae 5–8.

$$Accuracy = \frac{TP + TN}{TP + FN + FP + TN} \quad (5)$$

$$Precision = \frac{TP}{TP + FP} \quad (6)$$

$$Recall = \frac{TP}{TP + FN} \quad (7)$$

$$AUC = \int_0^1 \frac{TP}{P} d\frac{FP}{N} = \frac{1}{PN} \int_0^N TP dFP \quad (8)$$

Where, TP, FN, FP, and TN refer to True Positive, False Negative, False Positive, and True Negative, respectively. TP represents the number of correct classifications of positive observations of phenomena infected images; FN represents the number of incorrect classifications of positive observations (normal images) that are incorrectly labeled as phenomena infected images. FP represents the number of incorrect classifications of negative observations, and TN represents the number of correct classifications of negative observations.

4.3. Experiments

Three python IDEs are used in this study: *Jupyter notebook*, used to redistribute the dataset, *Kaggle*, used for implementation, and *Spyder* used for re-sampling.

Figures 7 and 8 show the ratios of the two types of images during training and validation after redistribution, and we note that the phenomenon of unbalanced distribution of data remains.

As for the third main stage in this research, data augmentation is used to obtain balanced ratios of images from both during the training and validation stages, as we see in Figs. 9 and 10.

Different number of layers are used for the two models in the feature extraction stage depending on the used optimizer. For instance, 85 layers are used for pre-activation ResNet and DenseNet169 used with the Adam optimizer, while 37 layers are used with the SGD optimizer. For the classification step, an ANN is used with one hidden layer containing 450 neurons and a Relu activation function, and one output layer with one neuron and a sigmoid activation function. These classes are designed for each of the three main stages: dealing with the original data, class weight, and re-sampling.

Parameter setting is one of the most sensitive steps and affects the results of the model, it is used to reduce network error. The values of these parameters are chosen experimentally for all experiments as presented in Table 4. Parameters' values are also related to the type of optimizer used, for instance, a learning rate of 0.0001 is used for the Adam optimizer for updating the network weights. 30 epochs are used which indicates the number of times the training dataset is divided according to the patch-size across the network, and accordingly the parameters in the network are updated. Default steps per epoch that are equal to several samples in the training dataset are divided by the batch-size, and default steps between validation epochs that are equal to several samples in testing dataset are divided by the batch-size for fitting the model with data step. L2 regularization is used with the SGD optimizer to overcome the overfitting problem, the learning rate is changed to 0.00001, and the number of epochs is also changed to 40; this value represents the number of forwards and backward stages to be used in the training and verification stages. Dropout regularization is used in the resampling stage instead of L2 regularization.

4.4. Experiment Results

In this section, we present the results and performance curves that express how the model performed during the training and validation phases. In the three main stages, original distribution stage, class weight, and re-sampling, the results of each stage are divided into sub-stages based on the type of optimizer used and other additional criteria. The metrics used for evaluation are precision, recall, accuracy, loss, and AUC. The model's performance in the training phase is represented in blue for all curves and the orange curve expresses the model's performance during the validation phase.

Table 4
Training Hyper-parameters

Parameter	Value
Batch size	24
Optimizer	Adam, SGD
Learning Rate	0.0001(Adam) and 0.00001(SGD)
Number of Epochs	30,40
Steps per epoch	Number of training samples / batch-size
Validation steps	Number of testing samples / batch-size
Regularization method	L2, Dropout

The performance of the model was very fluctuating and unstable, especially during the validation phase when using the Adam optimizer with the data in the original distribution, which is illustrated in Fig. 11.

When using the SGD optimizer and reducing the complexity of the model while using the data in its original distribution, the curve began to appear less kinked during the performance validation phase and is visible in Fig. 12.

The performance is still bad, even after conducting several experiments, the results have not been improved, and so we first had to increase the percentage of data during the validation stage to become 25% instead of 19%. Since the data is measured unbalanced, two methods are applied to overcome this problem, class weight and resampling.

When using the Adam optimizer with a class weight, the model's performance curves in the validation stage express the inability to generalize all the validation data, which is shown by the highly volatile curve in the validation stage shown in orange color in Fig. 13. Despite standing at excellent values at the end of the trip for all measures (recall of 99%, 0.01 of loss rate, and 97% for the rest of the measures) in the validation phase, the performance curves indicate that the performance is not guaranteed. This prompted us to implement another type of optimizer, which is the SGD, and its performance curves are illustrated in Fig. 13.

After applying the SGD optimizer with the class weight technique, the curves became significantly more stable for all metrics (Fig. 14). It ended with 0.97 for accuracy and AUC, and 0.24 for rate of loss, while performance was modest on the accuracy scale of 0.89 and recall of 0.88.

Metrics values and numbers improved when changing the weight of the least available samples in the data, which is the normal class, by 1.5 compared to 1.0 for the highest available samples. Although the curves are less stable than in the previous stage, as shown in Fig. 15, there were slightly better results as numbers, where the precision ratio reached 98%, and the stability of performance relative to the AUC

metric remained at 0.97, and the recall metric remained at 0.88, while the accuracy improved by 0.01. Its value becomes 90%, and the loss ratio becomes 0.23.

It found that the performance of the model after producing images with balanced ratios through data augmentation, especially when using the Adam optimizer with L2 regularization method was poor, as all performance curves in the validation stage were very jagged and unstable until the end (Fig. 16).

After using the SGD optimizer and reducing the number of model layers in addition to using a dropout of 0.8 which was chosen experimentally on the balanced data. The shape of the curves became more stable and less fluctuating, but there was little difference between the training curve and the validation, which is illustrated in Fig. 17.

We reduced the difference between the training curve and the validation curve by increasing the dropout ratio to 0.9 and the improvement is evident in Fig. 18.

4.5. Comparison results from all experiments

The experiments in this study are divided into three main stages. The first stage is to train the model on the data with its original distribution, and the remaining two stages deal with the imbalanced data in weighting stage and the resampling stage using data augmentation. The goal in this study is to reach a good performance numerically as well as behaviorally through its behavior in the performance curve so that we reach a stable model as much as possible. The results are presented through the performance curves in the previous paragraph in detail and Table 5 summarizes the results for each experiment.

The model obtained a bad and unstable performance when trained on the data with its original distribution. Referring to Table 5, it shows bad numerical results, so there was a need to increase the percentage of data in the performance validation stage and using techniques to deal with the irregular distribution in performance. The goal of this study is to make a trade-off and to combine the stability of the performance curve and the numerical results to choose the best results. Despite the high results of the class weight stage experiment with Adam optimizer, its performance curves were zigzag, which means that the model is not guaranteed performance. Therefore, the results of the different class weight experiment are nominated stage with SGD optimizer to combine good performance and acceptable shape in performance curves. We used these results to compare with related work in the next section.

4.6. Comparison with other approaches

The results of the proposed approach are compared with the results of nine previous approaches using the same dataset proposed in [17]. These results are presented in Table 6, Fig. 19, and Fig. 20. The proposed approach achieved promising results compared to other approaches especially for the precision metric. Comparing the results in terms of accuracy shows better results are achieved compared to many previous approaches, including ResNet-34 [15], MobileNetV2, CNN and LSTM from [29], the pre-trained Xception model [20], ResNet-50 [24], Xception, VGG19 and CNN [37]. While the difference was clear between our results and other models in measuring AUC, other models include MobileNetV2, CNN, LSTM

[29], the pre-trained Xception model [20], ResNet-50 [24], the results are also summarized in Fig. 23. The last column shows the difference between our results in the loss scale compared to ResNet-34 [15], [27] and its four algorithms [23], [37] (Figs. 21–23).

Table 5
The comparison results of our experiments

Stage	Result stage	precision	Recall	Accuracy	AUC	Loss
Original distribution stage with Adam optimizer	Training result	0.74	0.99	0.74	0.53	4.42
	validation result	0.62	0.99	0.62	0.54	6.10
Original distribution stage with SGD optimizer	Training result	0.96	0.95	0.94	0.98	0.14
	validation result	0.90	0.92	0.88	0.94	0.27
Class weight Stage with Adam optimizer	Training result	0.99	0.99	0.99	0.99	0.002
	validation result	0.97	0.99	0.97	0.97	0.01
Equal class weight Stage with SGD optimizer	Training result	0.95	0.93	0.92	0.97	0.19
	validation result	0.97	0.88	0.89	0.97	0.24
Different class weight Stage with SGD optimizer	Training result	0.96	0.93	0.92	0.97	0.18
	validation result	0.98	0.88	0.90	0.97	0.23
Re-sampling Stage with SGD optimizer and dropout of 0.8	Training result	0.974	0.939	0.957	0.992	0.115
	validation result	0.880	0.890	0.883	0.953	0.279
Re-sampling Stage with SGD optimizer and dropout of 0.9	Training result	0.965	0.927	0.946	0.987	0.142
	validation result	0.876	0.907	0.888	0.948	0.292

Table 6
Comparison results with others works

Reference	Algorithm	Accuracy	Recall	Precision	AUC	Loss
[2]	CNN model	84	-	-	-	0.8
[3]	CNN	78	-	-	-	-
[15]	ResNet-34	92	99	90	-	1.6
[27]	VGG16	87	96	-	-	0.3
	VGG19	88	95	-	-	0.3
	Inception V3	70	84	-	-	0.9
	ResNet50	77	97	-	-	0.6
[29]	ResNet152V2	99	99	99	99	-
	MobileNetV2	96	99	95	97	-
	CNN	92	92	95	96	-
	LSTM	91	92	93	95	-
[20]	Xception pre-trained model	-	99	84	96	0.04
[24]	Pre-trained ResNet-50	90	93	93	89	0.03
[23]	VGG16	87	-	-	-	0.3
	Xception	82	-	-	-	0.45
[37]	Xception	83	-	95	-	0.6
	DenseNet201	93	-	99	-	1.9
	MobileNet_v2	96	-	98	-	0.24
	VGG19	85	-	80	-	1.3
	CNN	84	-	94	-	0.4
	VGG16	86	-	87	-	1
	Inception_v3	94	-	93	-	1.76
	ResNet 50	96	-	98	-	1.5
	Inception_ResNet_V2	96	-	98	-	1.1
The proposed model (different class weight stage) with SGD)	Pre-activation ResNet with DenseNet169	90	88	98	97	0.23

4.7. Results Discussion

The proposed approach was able to reach a performance that combines good results numerically and behaviorally acceptable in the performance curve. In addition, we overcame many challenges that encountered during this study, like the small size of the test data, which appeared for the first time in the performance curve. To meet this challenge, we repartitioned the training and validation data from scratch. And then we used several techniques to deal with the imbalance of data distribution, which is the categorical weight and re-sampling with the increase of the data. Another challenge was the problem of overfitting, which we addressed by reducing the number of ResNet pre-activation layers and using an organization type suitable for the two techniques implemented in the previous challenge. Where L2 is used with the class weighing technique, and the dropout is best suited with the re-sampling technique. In addition to changing the optimizer from Adam to SGD, where the use of the SGD optimizer had a positive effect in terms of model performance stability compared to the results obtained when applying the Adam optimizer. The explanation for this phenomenon is that the SGD optimizer splits the batch number of features equal to the number of that features, making the performance movement more stable, specifically in the loss scale, and thus, the possibility of updating the weights for each training sample. The second conclusion is that the method for dealing with unbalanced data has a role in determining the type of regularization method that is best, where the L2 method was the best suited for the class weight technique, while the dropout method was the best suited for the re-sampling technique, which may be due to the volume of data used in each stage. The results were very good on the precision, AUC, and loss metrics. However, the results were modest in terms of performance on measures of accuracy and recall. The reason according to our analysis was the inequality of weight for each class, so it may cause a conflict in the classification of the two classes. When comparing our results with those of other models, we got good results on most metrics, except for recall (Figs. 19–23). Finally, we can also say that the proposed approach was able to outperform many previous studies in terms of reaching stable performance and among these studies that reached fluctuating performance regardless of the results as the values [22–24], [27], and [44]. In addition, the results have a small loss rate, and the difference was obvious compared to the two studies in [26] and [2], which had an error rate of more than 1.00.

5. Conclusion

AI applications have proven their efficiency in the early diagnosis of many diseases, which contributed to saving the lives of many patients. In this study, we proposed an approach by merging different deep learning models to extract features from CXR images for the automatic detection of Pneumonia disease. Two models are proposed: pre-activation ResNet and pre-trained DenseNet169. Most of the previous researchers focused on highlighting numerical results without considering the estimate of performance in the performance curve in the training and validation stages, while our focus in this research was more on combining the stability of the performance curve in addition to the numerical value of the performance measures. The proposed approach has obtained a well-stable curve compared with many previous studies [19][22]. As numerical results, we achieved a precision of 98%, AUC of 97%, and a loss of 23%,

while the performance was modest in the two metrics of accuracy. From the findings of the proposed approach, we conclude the following points:

- The nature and distribution of data have a significant impact on the performance of the model, and each type of data has its own appropriate ways to deal with.
- The type of optimizer used may increase the quality and stability of the model's performance or vice versa. It was evident in the results that it became more stable when applying the SGD optimizer compared to its performance with the Adam optimizer.
- Reducing the number of layers or blocks in the pre-activation ResNet model contributes to stability and better results as well.
- The more stable the performance curve, the more assured its performance.
- Evaluating the performance of the model based on the final results of the performance measures as numbers alone are not sufficient to prove the quality and effectiveness of the model.
- The proposed approach was able to outperform many previous studies to which we dedicated a special section.
- Since the size of the proposed dataset was not sufficient to achieve satisfactory results, we had to redistribute it again; the distributions were also unbalanced, and we handled it in two ways: category weighting and re-sampling as the data increased.
- The proposed approach was able to obtain more stable performance curve than many previous works, which are detailed in the discussion section.

For future work, we suggest applying the proposed approach to data on COVID-19 disease, and to integrate ResNet pre-activation with other CNN models.

Declarations

Ethics approval

This article does not contain any studies with human participants or animals performed by any authors.

Consent to participate

Not applicable.

Competing interests

The authors declare that they have no competing interests. All authors certify that they have no affiliations with or involvement in any organization or entity with any financial interest or non-financial

interest in the subject matter or materials discussed in this manuscript.

Authors' contributions

Both authors wrote and read the manuscript, A.T.A reviewed, revised, prepared, and approved the final manuscript for publication.

Funding

Not applicable.

Availability of data and materials

This publication is supported by an online dataset which is openly available at:
<https://doi.org/10.1016/j.cell.2018.02.010>.

Acknowledgements

Not applicable.

References

1. Chouhan V, Singh SK, Khamparia A, Gupta D, Tiwari P, Moreira C, Damaševičius R, de Albuquerque VHC (2020). A Novel Transfer Learning Based Approach for Pneumonia Detection in Chest X-ray Images. *Applied Sciences*, 10(2), 559. <https://doi.org/10.3390/app10020559>.
2. Sharma, A, Negi, M, Goyal, A, Jain, R, Nagrath, P (2021). Detection of pneumonia using ML & DL in python. *IOP Conference Series: Materials Science and Engineering*, 1022, 1st International Conference on Computational Research and Data Analytics (ICCRDA 2020) 24th October 2020, Rajpura, India. <https://doi.org/10.1088/1757-899X/1022/1/012066>.
3. Saul, CJ, Urey, DY, Taktakoglu, CD (2019). Early Diagnosis of Pneumonia with Deep Learning. <https://doi.org/10.48550/arXiv.1904.00937>.
4. Sarkar, R, Hazra, A, Sadhu, K, Ghosh, P (2020). A Novel Method for Pneumonia Diagnosis from Chest X-Ray Images Using Deep Residual Learning with Separable Convolutional Networks. In: Gupta, M., Konar, D., Bhattacharyya, S., Biswas, S. (eds) *Computer Vision and Machine Intelligence in Medical Image Analysis. Advances in Intelligent Systems and Computing*, 992. Springer, Singapore. https://doi.org/10.1007/978-981-13-8798-2_1.
5. Tilve, A, Nayak, S, Vernekar, S, Turi, D, Shetgaonkar, PR, Aswale, S. (2020). Pneumonia Detection Using Deep Learning Approaches. *International Conference on Emerging Trends in Information Technology and Engineering (ic-ETITE)*, pp. 1-8, <http://doi.org/10.1109/ic-ETITE47903.2020.152>.
6. Varshni D, Thakral K, Agarwal L, Nijhawan R, Mittal A (2019). Pneumonia Detection Using CNN based Feature Extraction, *IEEE International Conference on Electrical, Computer and Communication Technologies (ICECCT)*, pp. 1-7, <http://doi.org/10.1109/ICECCT.2019.8869364>.

7. Alqudah, AM, Qazan, S, Masad, IS (2021). Artificial Intelligence Framework for Efficient Detection and Classification of Pneumonia Using Chest Radiography Images. *J. Med. Biol. Eng.* 41, 599–609. <https://doi.org/10.1007/s40846-021-00631-1>.
8. Jaiswal, AK, Tiwari, P, Kumar, S, Gupta, D, Khanna, A, Rodrigues, J (2019). Identifying pneumonia in chest X-rays: A deep learning approach. *Measurement*, 145, 511–518. <https://doi.org/10.1016/j.measurement.2019.05.076>.
9. Kido S, Ikezoe J, Naito H, Tamura S, Machi S. Fractal analysis of interstitial lung abnormalities in chest radiography (1995). *Radiographics*, 15(6), 1457-64. <http://doi.org/10.1148/radiographics.15.6.8577968>.
10. Ishida, T, Katsuragawa, S, Ashizawa, K, MacMahon, H, Doi, K (1998). Application of artificial neural networks for quantitative analysis of image data in chest radiographs for detection of interstitial lung disease. *Journal of Digital Imaging*, 11(4), 182–192. <https://doi.org/10.1007/BF03178081>.
11. Rafael T, Sousa, Oge M, Fabrizio Alphonsus AMN, Soares Iwens IG, Sene Leandro L.G. de Oliveira, Edmundo SS (2013). Comparative Performance Analysis of Machine Learning Classifiers in Detection of Childhood Pneumonia Using Chest Radiographs, *Procedia Computer Science*, 18, 2579-2582, <https://doi.org/10.1016/j.procs.2013.05.444>.
12. Reza S, Amin OB, Hashem MMA (2019). A Novel Feature Extraction and Selection Technique for Chest X-ray Image View Classification. 5th International Conference on Advances in Electrical Engineering (ICAEE), pp. 189-194. <http://doi.org/10.1109/ICAEE48663.2019.8975457>.
13. Verma G, Prakash S (2020). Pneumonia Classification using Deep Learning in Healthcare. *International Journal of Innovative Technology and Exploring Engineering (IJITEE)*, 9(4), 1715–1723. <https://doi.org/10.35940/ijitee.d1599.029420>.
14. Ansari N, Faizabadi A, & Motakabber S, Ibrahimy M (2020). Effective Pneumonia Detection using ResNet based Transfer Learning. *Test Engineering and Management*. 82, 15146 - 15153.
15. Kadam K, Ahirrao S, Kaur H, Shraddha Ph, Pawar, A. (2019). Deep Learning Approach for Prediction of Pneumonia. *International Journal of Scientific and Technology Research*, 8(10), 2986–2989.
16. Yoo S, Gujrathi I, Haider MA, Khalvati F (2019). Prostate Cancer Detection using Deep Convolutional Neural Networks. *Scientific Reports*, 9, 19518. <https://doi.org/10.1038/s41598-019-55972-4>.
17. Kermany, DS, Goldbaum M, Cai W, Valentim CCS, et al. (2018). Identifying Medical Diagnoses and Treatable Diseases by Image-Based Deep Learning. *Cell*, 172(5), 1122-1131.e9. <https://doi.org/10.1016/j.cell.2018.02.010>.
18. Toğaçar M, Ergen B, Cömert Z, Özyurt F (2020). A Deep Feature Learning Model for Pneumonia Detection Applying a Combination of mRMR Feature Selection and Machine Learning Models. *IRBM*, 41(4), 212–222. <https://doi.org/10.1016/j.irbm.2019.10.006>.
19. Almaslukh B (2021). A Lightweight Deep Learning-Based Pneumonia Detection Approach for Energy-Efficient Medical Systems. *Wireless Communications and Mobile Computing*, 2021. <https://doi.org/10.1155/2021/5556635>.

20. Luján-García, JE, Yáñez-Márquez C, Villuendas-Rey Y, Camacho-Nieto O. (2020). A Transfer Learning Method for Pneumonia Classification and Visualization. *Applied Sciences*, 10(8), 2908. <https://doi.org/10.3390/app10082908>.
21. Ben Atitallah S, Driss M, Boulila W, Koubaa A, ben Ghézala H (2022). Fusion of convolutional neural networks based on Dempster–Shafer theory for automatic pneumonia detection from chest X-ray images. *International Journal of Imaging Systems and Technology*, 32(2), 658–672. <https://doi.org/10.1002/ima.22653>.
22. Islam KhT, Wijewickrema S, Collins, AM, O'Leary S (2020). A Deep Transfer Learning Framework for Pneumonia Detection from Chest X-ray Images. 286-293. 10.5220/0008927002860293. In: *Proceedings of the 15th International Conference on Computer Vision Theory and Applications*. <http://doi.org/10.5220/0008927002860293>.
23. Ayan E., Ünver HM (2019). Diagnosis of pneumonia from Chest X-ray Images Using Deep Learning. *2019 Scientific Meeting on Electrical-Electronics & Biomedical Engineering and Computer Science (EBBT)*, pp. 1-5. <http://doi.org/10.1109/EBBT.2019.8741582>.
24. Hussain SMH, Raju SM, Ismail AR (2021). Predicting Pneumonia and Region Detection from X-Ray Images using Deep Neural Network, arXiv preprint, *arXiv:2101.07717*. <https://doi.org/10.48550/arXiv.2101.07717>.
25. Singh S, Tripathi BK (2022). Pneumonia classification using quaternion deep learning. *Multimedia Tools and Applications*, 81, 1743–1764. <https://doi.org/10.1007/s11042-021-11409-7>.
26. Račić L, Popović T, čakić S, Šandi, S. (2021). Pneumonia Detection Using Deep Learning Based on Convolutional Neural Network. 25th International Conference on Information Technology (IT), pp. 1-4. <http://doi.org/10.1109/IT51528.2021.9390137>.
27. Jain R, Nagrath P, Kataria G, Kaushik VS, Jude Hemanth D (2020). Pneumonia detection in chest X-ray images using convolutional neural networks and transfer learning. *Measurement*, 165, 108046. <https://doi.org/10.1016/j.measurement.2020.108046>.
28. Shorten C, Khoshgoftaar, TM (2019) A survey on Image Data Augmentation for Deep Learning, *Journal of Big Data*, 60, <https://doi.org/10.1186/s40537-019-0197-0>.
29. Elshennawy NM, Ibrahim DM (2020) 'Deep-Pneumonia Framework Using Deep Learning Models Based on Chest X-Ray Images', *Diagnostics*, 10(9), 649. <http://doi.org/10.3390/diagnostics10090649>.
30. He K, Zhang X, Ren S, Sun J (2016). Identity Mappings in Deep Residual Networks. In: Leibe B, Matas J, Sebe N, Welling M (eds), *Computer Vision – ECCV 2016*, Lecture Notes in Computer Science, vol. 9908. Springer, Cham. https://doi.org/10.1007/978-3-319-46493-0_38.
31. He K, Zhang X, Ren Sh, Sun J (2016). Deep residual learning for image recognition. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770-778. <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=7780459>.
32. Shang W, Chiu J., and Sohn K (2017). Exploring Normalization in Deep Residual Networks with Concatenated Rectified Linear Units (AAAI-17). In: *Proceedings of the Thirty-First AAAI Conference on*

- Artificial Intelligence, 31(1), 509-1516. <https://doi.org/10.1609/aaai.v31i1.10759>.
33. Bjorck J, Gomes C, Selman B, Weinberger KQ (2018). Understanding Batch Normalization, [Online], Available at: <https://arxiv.org/abs/1806.02375>. Last Accessed 31/10/2022.
 34. Khan, A., Sohail, A., Zahoor, U., & Qureshi, A. S. (2020). A survey of the recent architectures of deep convolutional neural networks. *Artificial intelligence review*, 53, 5455-5516. <https://doi.org/10.1007/s10462-020-09825-6>.
 35. Desarda A (2020), Build a Custom ResNetV2 with the desired depth from scratch, [Online], Available at: <https://towardsdatascience.com/build-a-custom-resnetv2-with-the-desired-depth-92892ec79d4b>, Last Accessed 31/10/2022.
 36. Huang G, Liu Z, Van Der Maaten L, Weinberger KQ (2017). Densely connected convolutional networks. In: *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, pp. 2261-2269. <https://doi.org/10.1109/CVPR.2017.243>.
 37. Kundu R, Das R, Geem ZW, Han G-T, Sarkar R (2021). Pneumonia detection in chest X-ray images using an ensemble of deep learning models. *PLoS ONE*, 16(9): e0256630 <https://doi.org/10.1371/journal.pone.0256630>.
 38. El Asnaoui K, Chawki Y, Idri A (2021). Automated Methods for Detection and Classification Pneumonia Based on X-Ray Images Using Deep Learning. In: Maleh Y, Baddi Y, Alazab M, Tawalbeh L, Romdhani I (eds), *Artificial Intelligence and Blockchain for Future Cybersecurity Applications. Studies in Big Data*, vol 90. Springer, Cham. https://doi.org/10.1007/978-3-030-74575-2_14.
 39. Wu Q, Wang F (2019) Concatenate Convolutional Neural Networks for Non-Intrusive Load Monitoring across Complex Background', *Energies*, 12(8), 1572. <http://doi.org/10.3390/en12081572>.
 40. Brownlee J (2021). How to Choose an Activation Function for Deep Learning. [Online], Available at: <https://machinelearningmastery.com/choose-an-activation-function-for-deep-learning/>, last accessed 31/10/2022.
 41. Collis J (2017). Glossary of Deep Learning: Bias. [Online], Available at: <https://medium.com/deeper-learning/glossary-of-deep-learning-bias-cf49d9c895e2>, Last Accessed 31/10/2022.
 42. Mooney P (2018), Chest X-Ray Images (Pneumonia), [online], Available at: <https://www.kaggle.com/paultimothymooney/chest-xray-pneumonia>, Last Accessed 31/10/2022.
 43. Vakili M, Ghamsari M, Rezaei M (2020). Performance Analysis and Comparison of Machine and Deep Learning Algorithms for IoT Data Classification. *arXiv:2001.09636v1*, <https://doi.org/10.48550/arXiv.2001.09636>.
 44. Labhane G, Pansare R, Maheshwari S, Tiwari R, Shukla A (2020). Detection of Pediatric Pneumonia from Chest X-ray Images using CNN and Transfer Learning. *3rd International Conference on Emerging Technologies in Computer Engineering: Machine Learning and Internet of Things (ICETCE)*, pp. 85-92. <http://doi.org/10.1109/ICETCE48199.2020.9091755>.

Figures



Figure 1

Normal, bacterial, and pneumonia images [17]

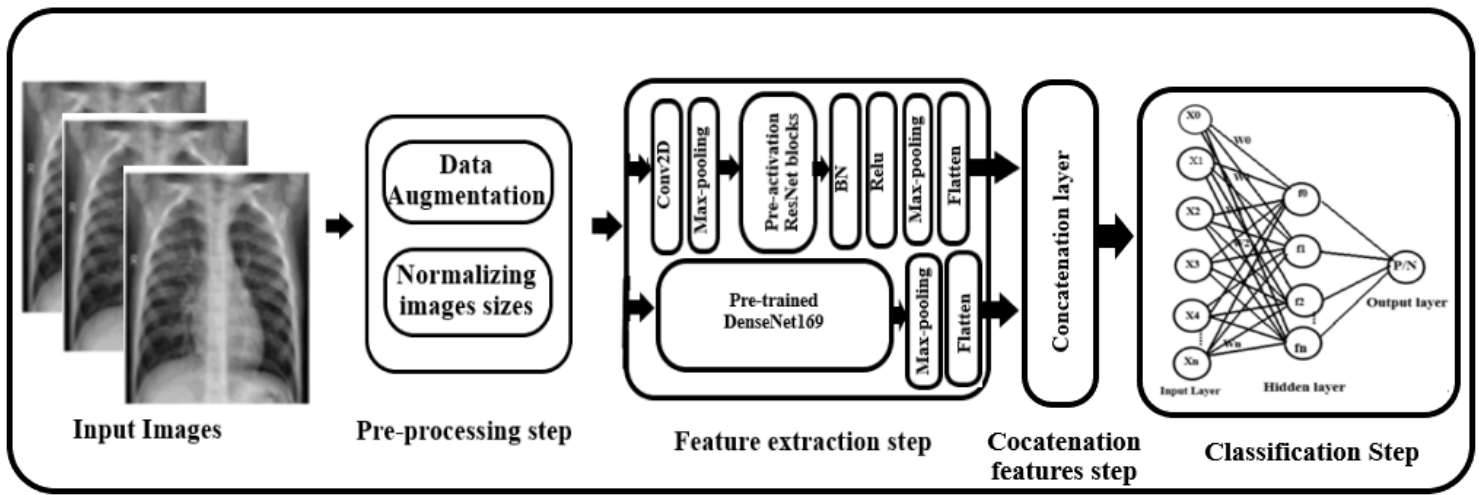
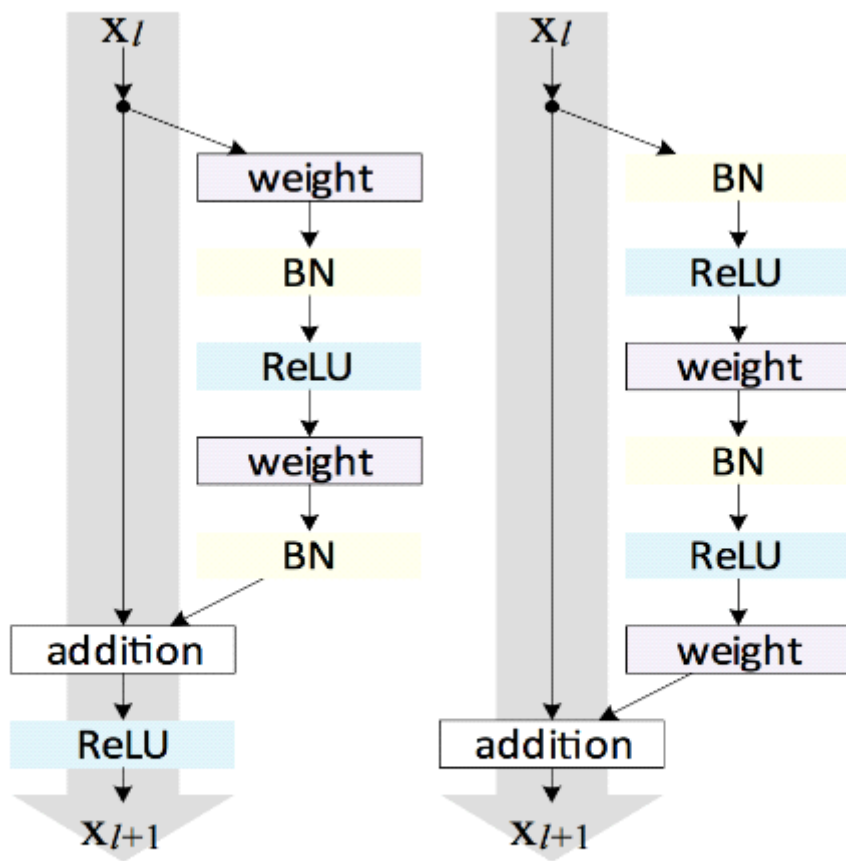


Figure 2

The proposed methodology



(a) original

(b) proposed

Figure 3

ResNet VS Pre-activation ResNet [30]

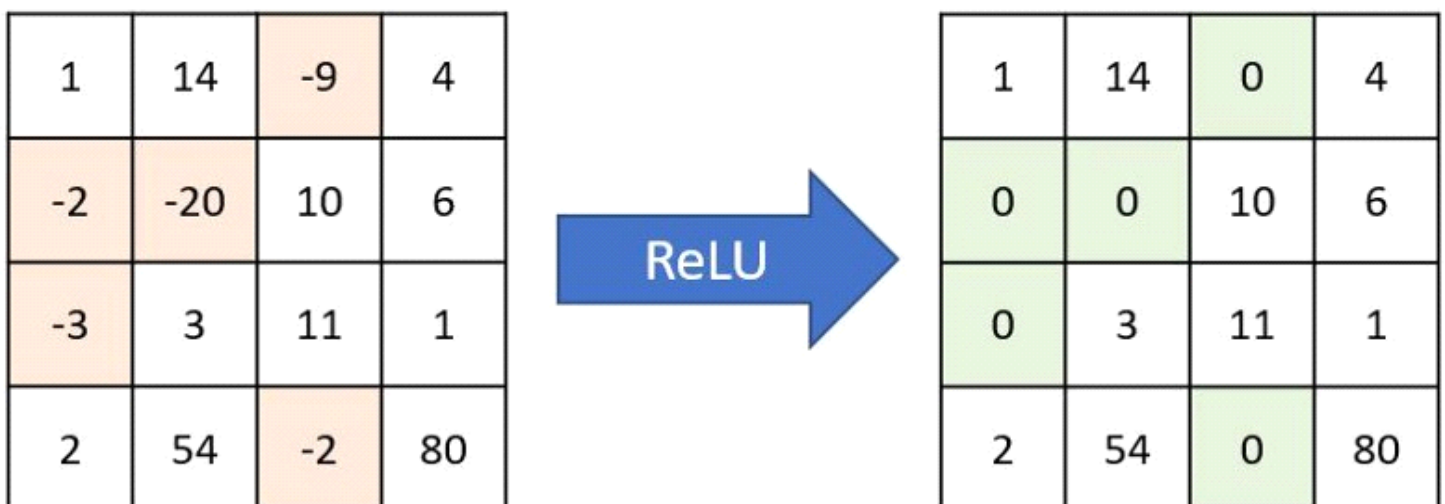


Figure 4

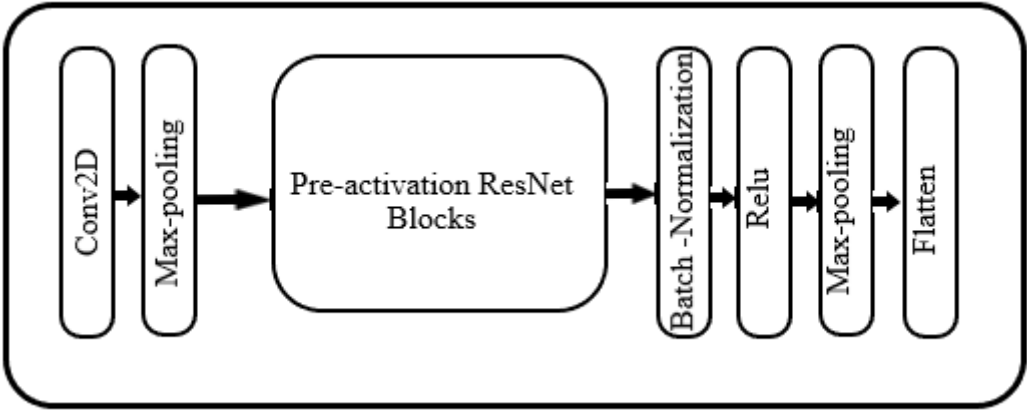


Figure 5

The proposed pre-activation ResNet.

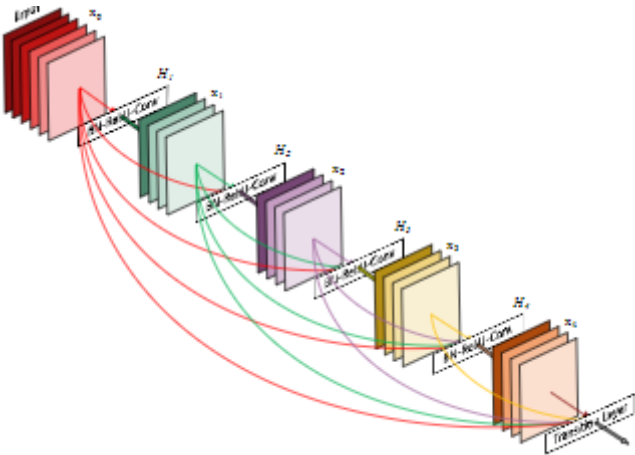


Figure 6

DenseNet 5 Layer Connection [35]

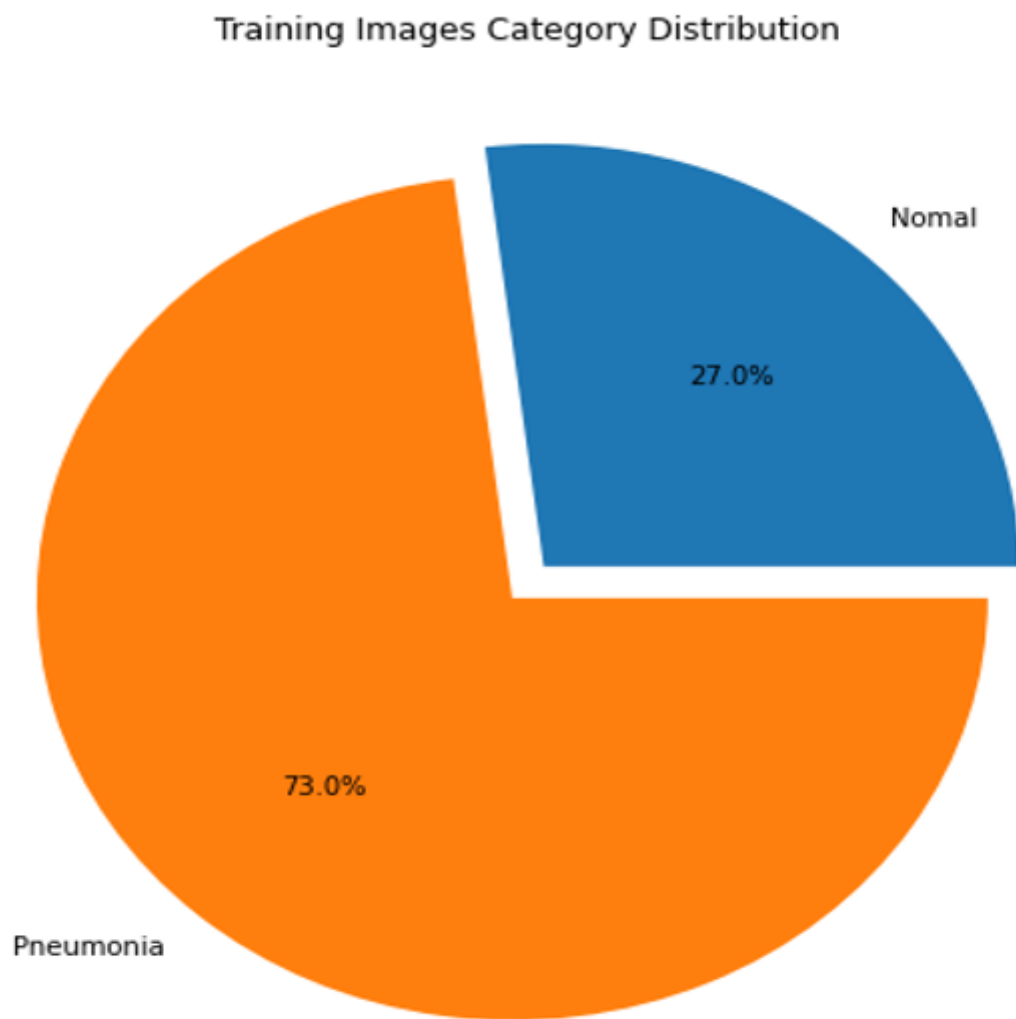


Figure 7

Distribution of two classes in the training data after re-distribute data

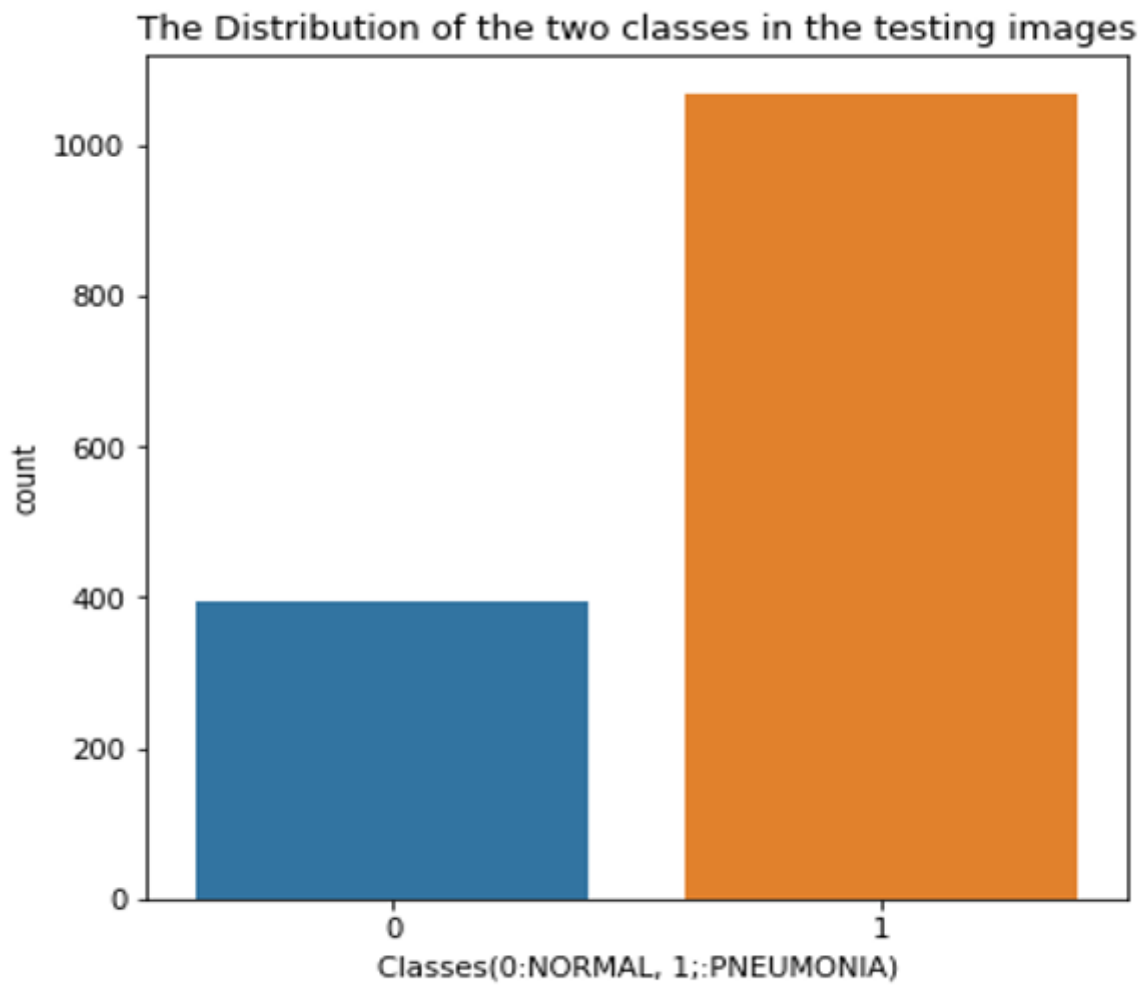


Figure 8

Distribution of two classes in the validation data

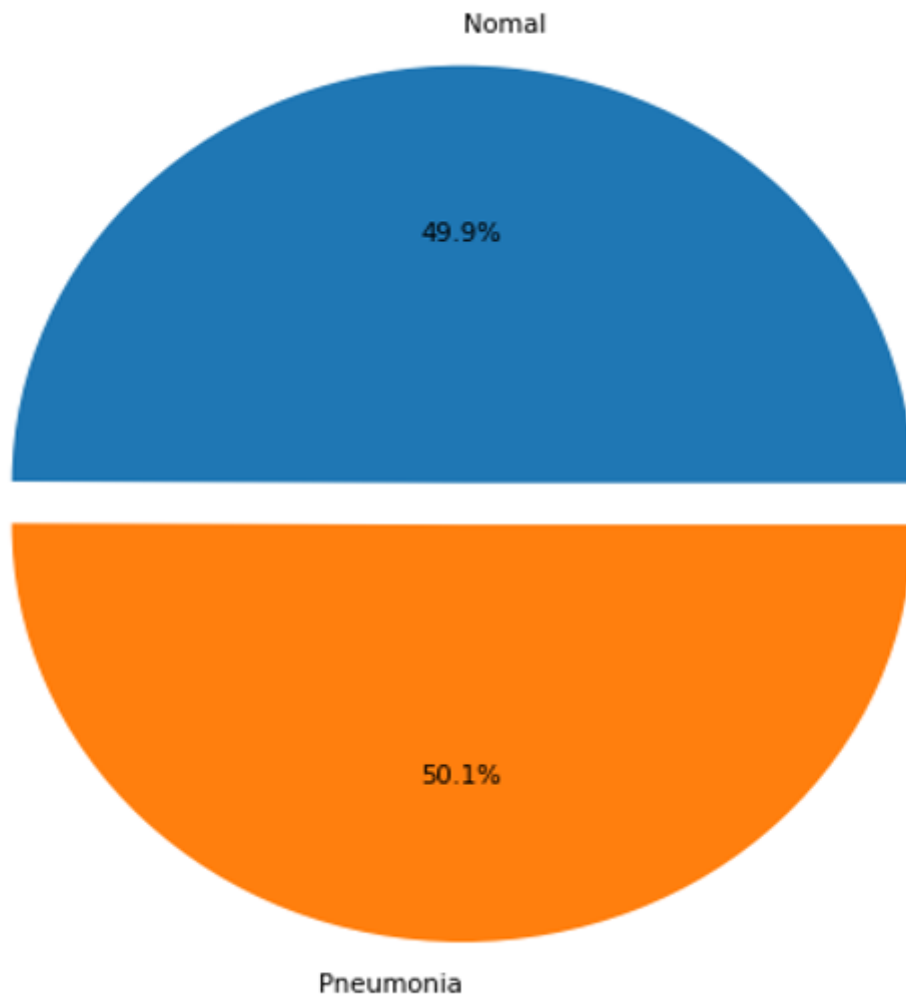


Figure 9

Distribution of two classes in the training data after re-sampling data

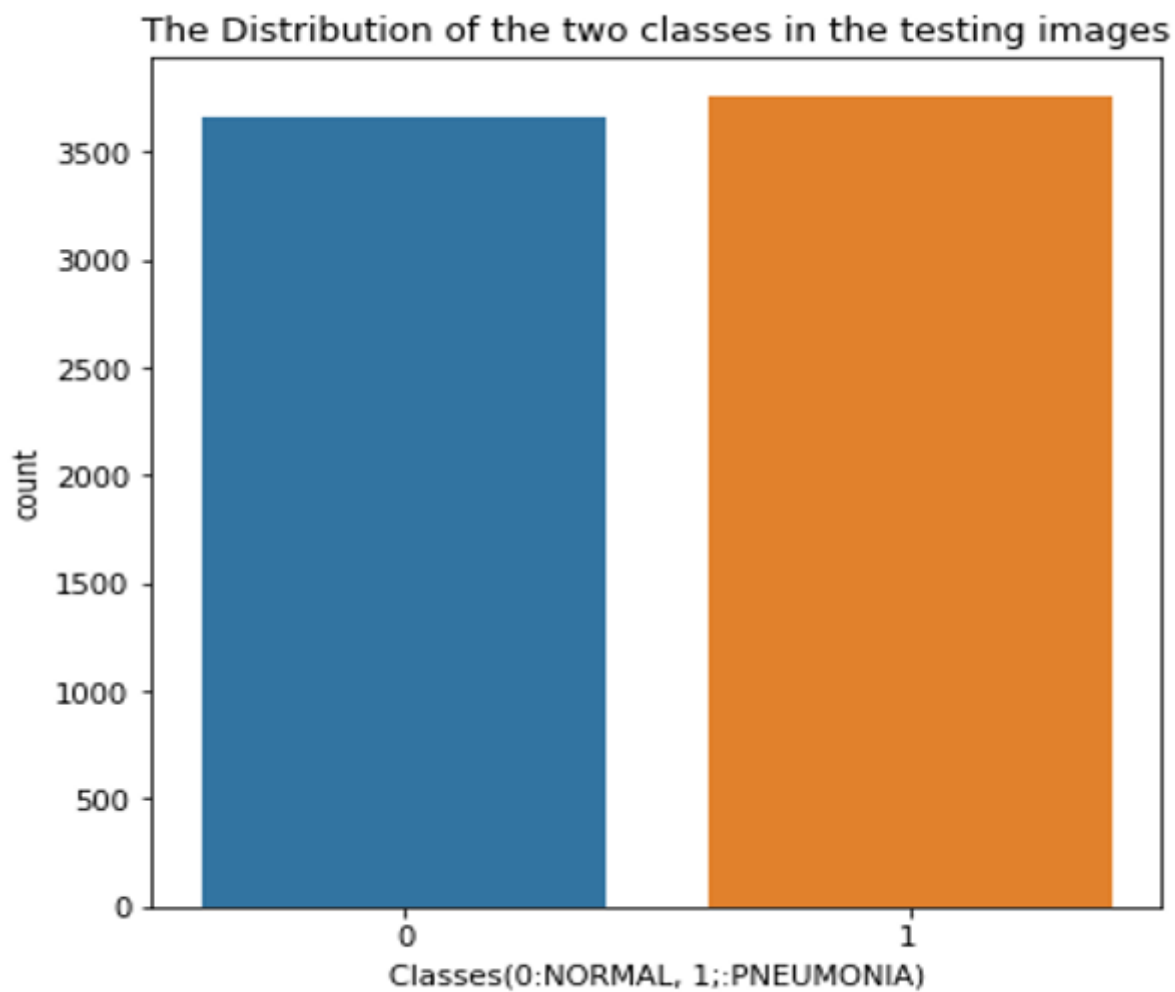


Figure 10

Distribution of two classes in the validation data after re-sampling data

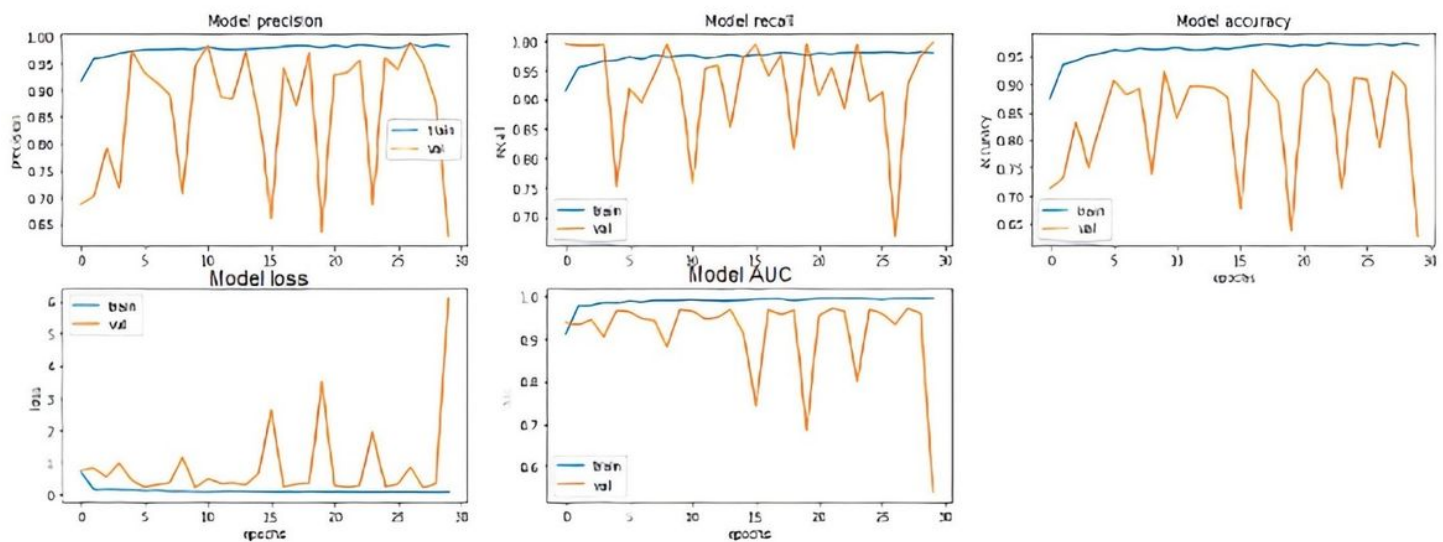


Figure 11

Performance curves from the original dataset stage with Adam optimizer

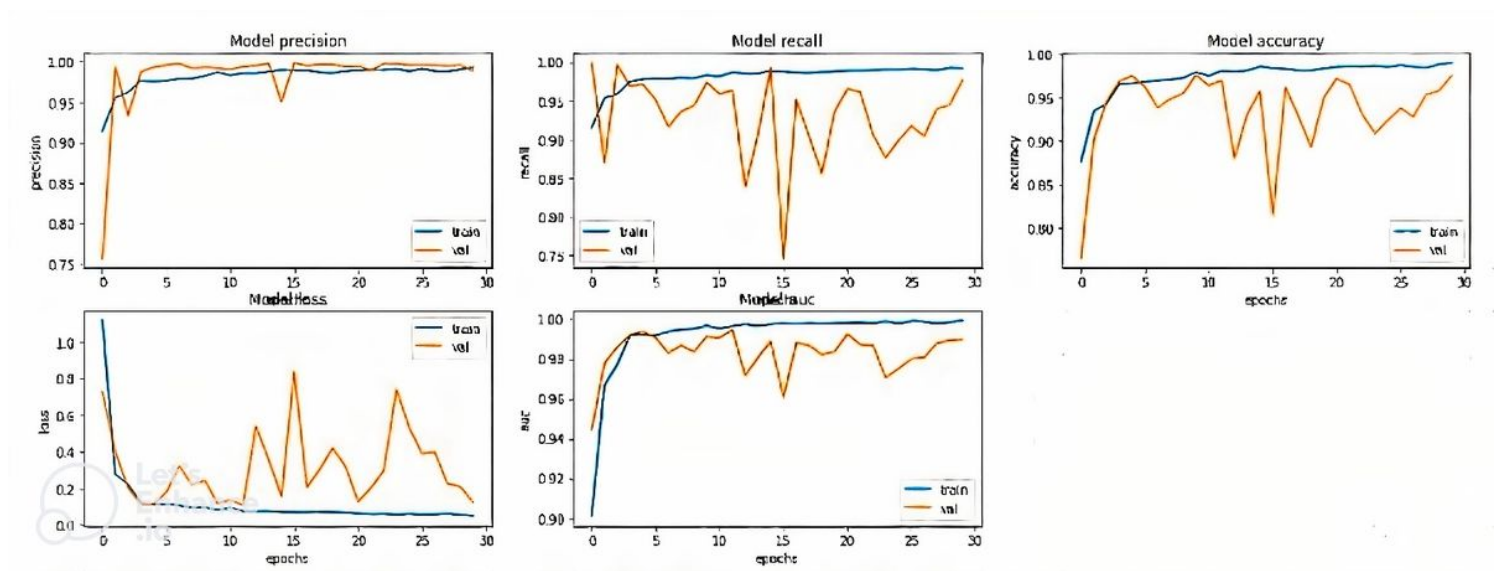


Figure 13

Performance curves after applying the class weight and Adam optimizer

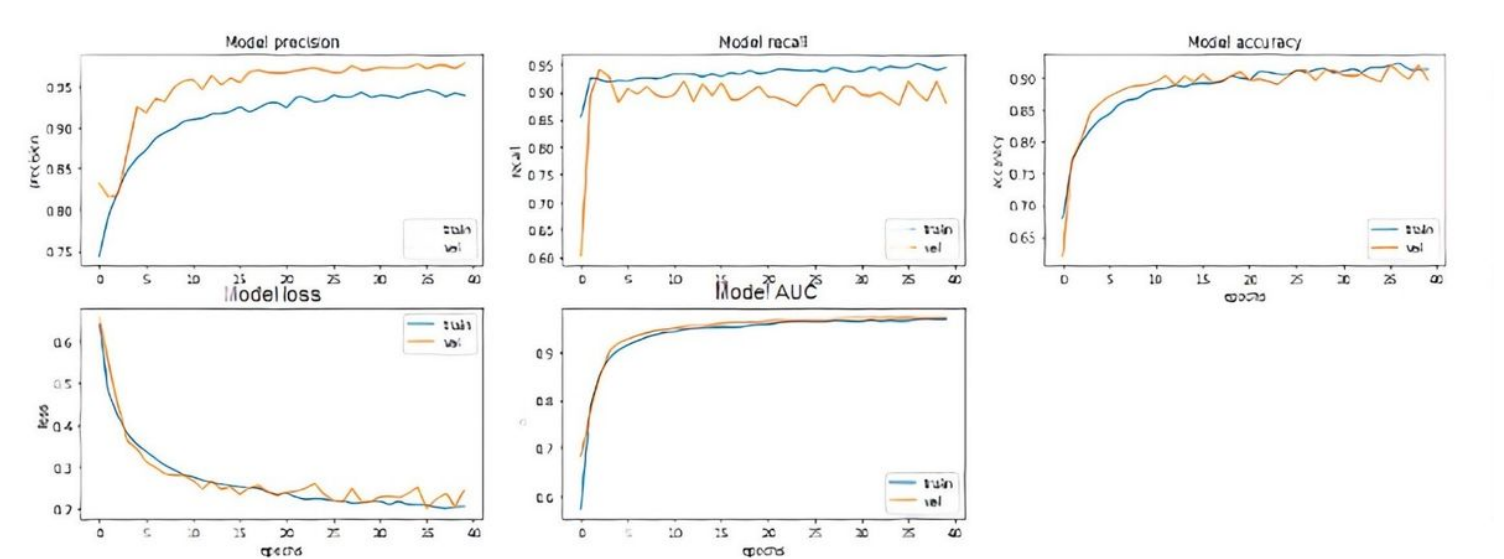


Figure 14

Performance curves after applying the class weight and SGD optimizer

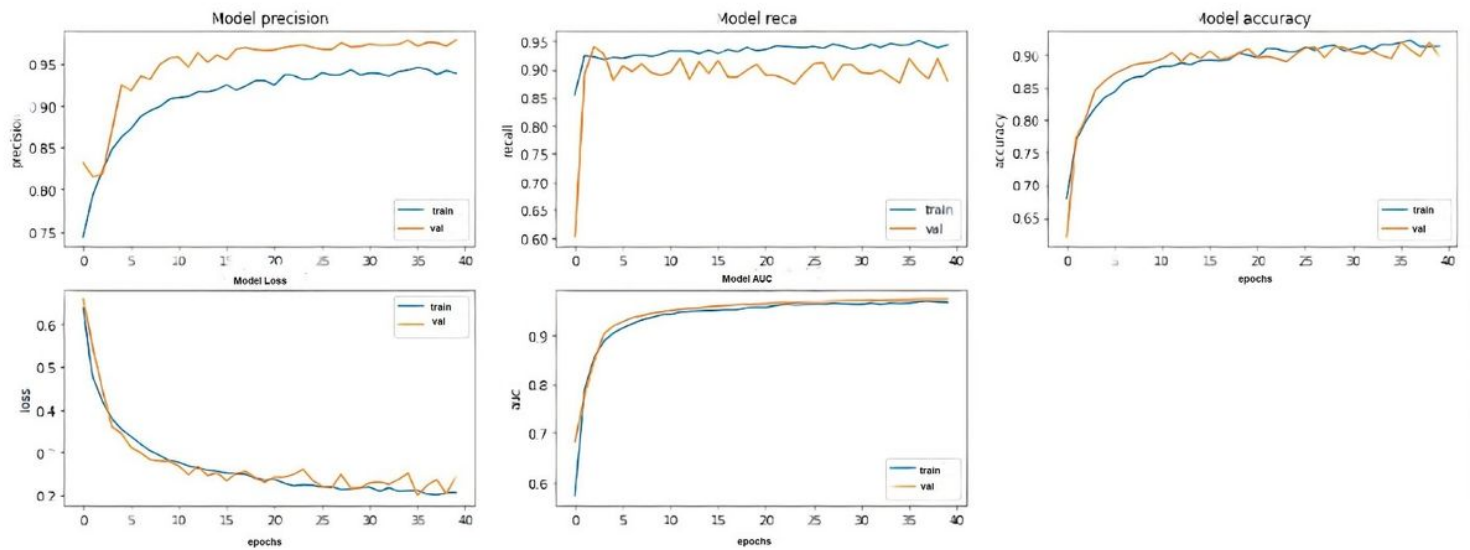


Figure 15

performance curves from different class weights and SGD

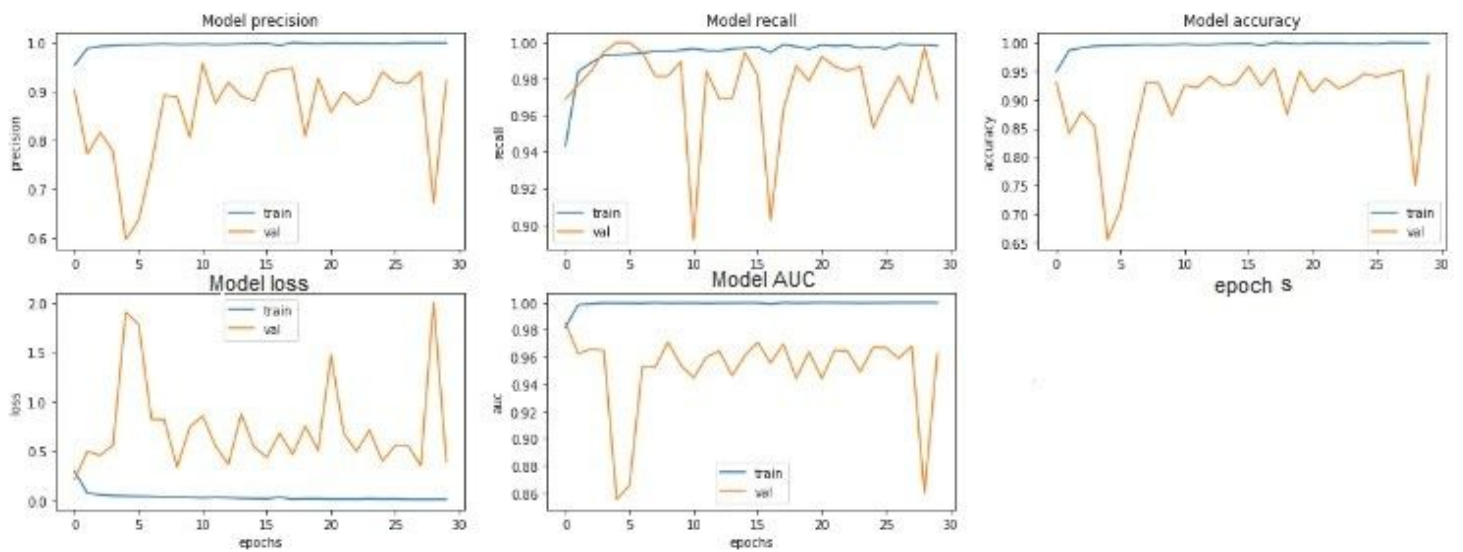


Figure 16

Performance curves from the re-sampling stage with Adam optimizer

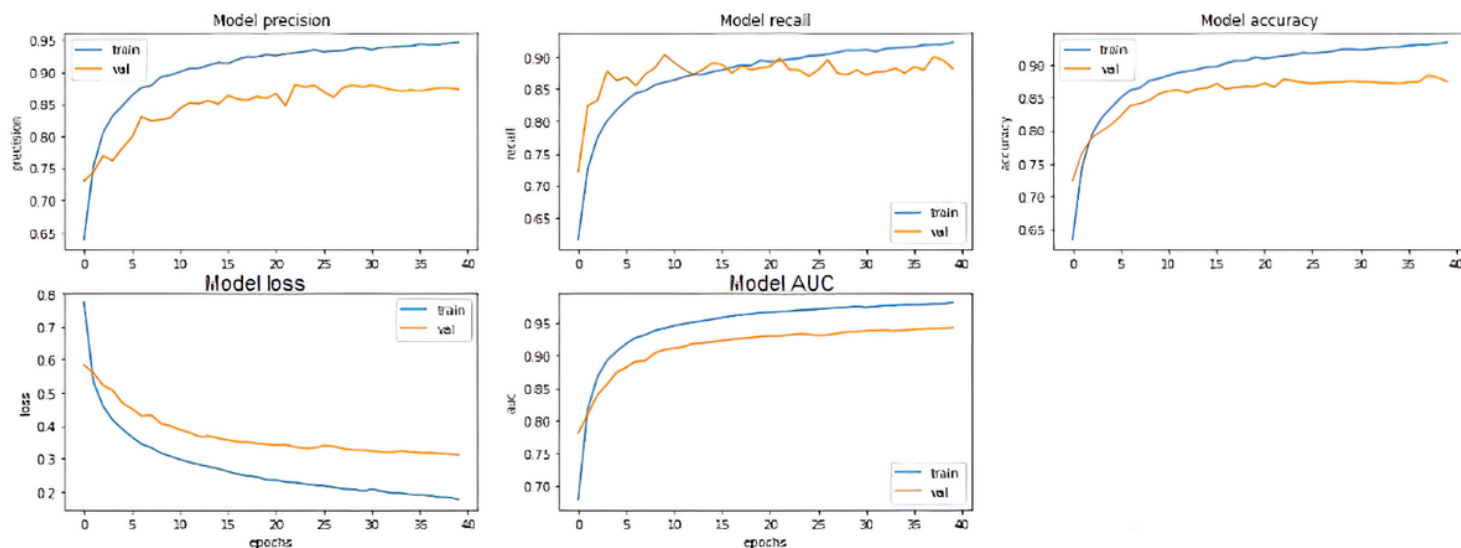


Figure 17

The performance curves from the re-sampling stage, SGD and a dropout of 0.8

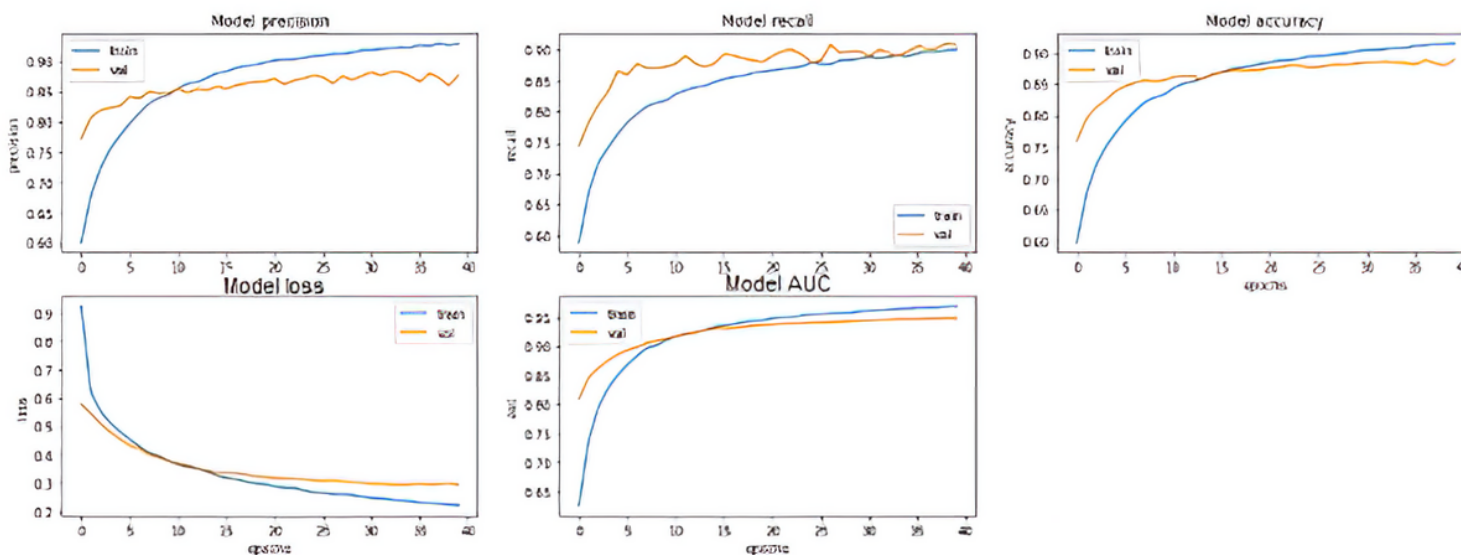


Figure 18

The performance curves from the re-sampling, SGD, and a dropout of 0.9

Accuracy

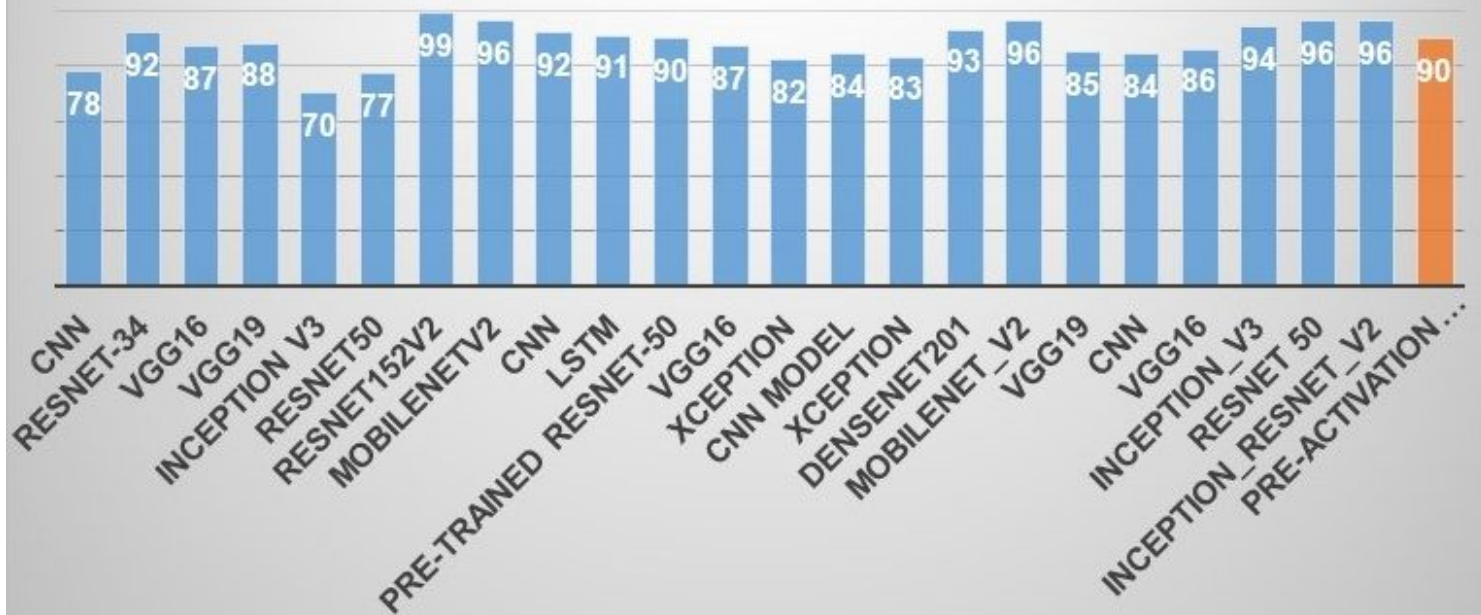


Figure 19

A comparison results with the Accuracy metric

Recall

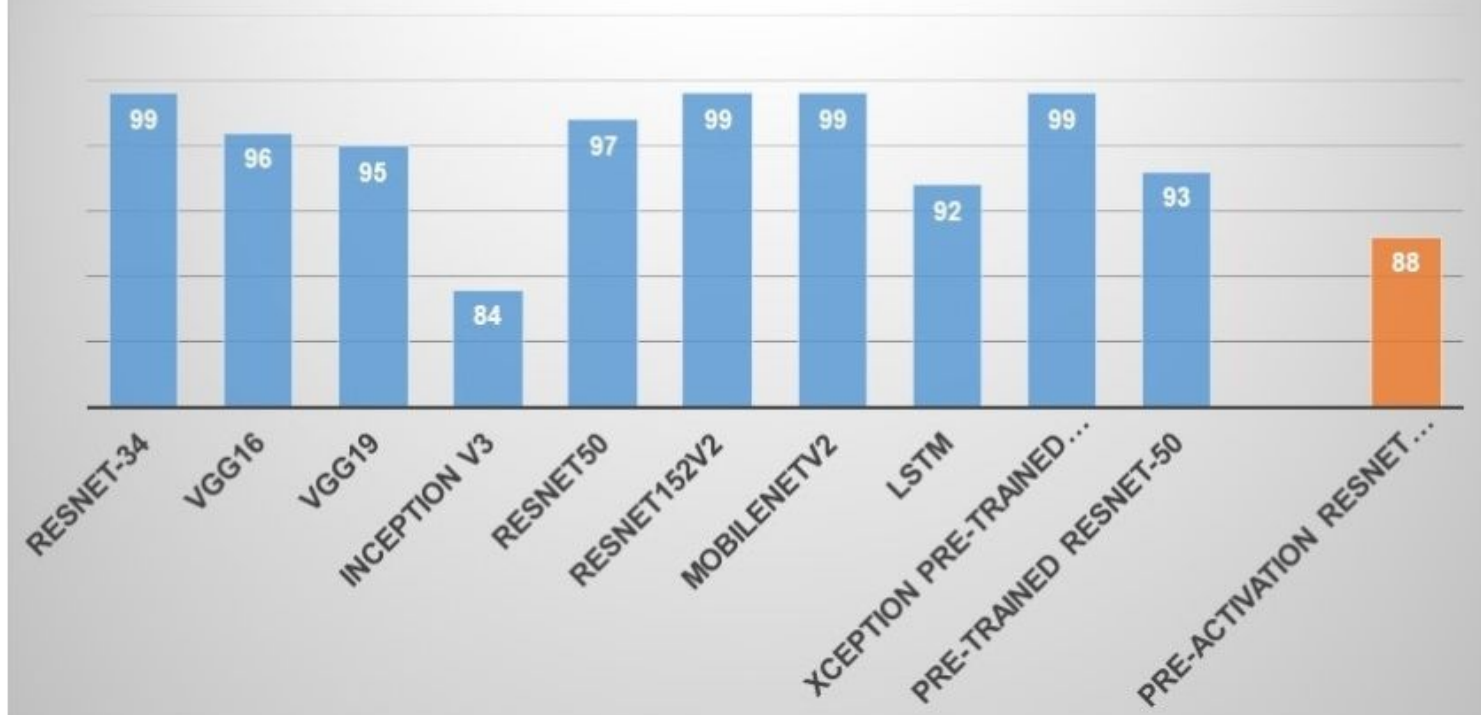


Figure 20

A comparison results with the Recall metric

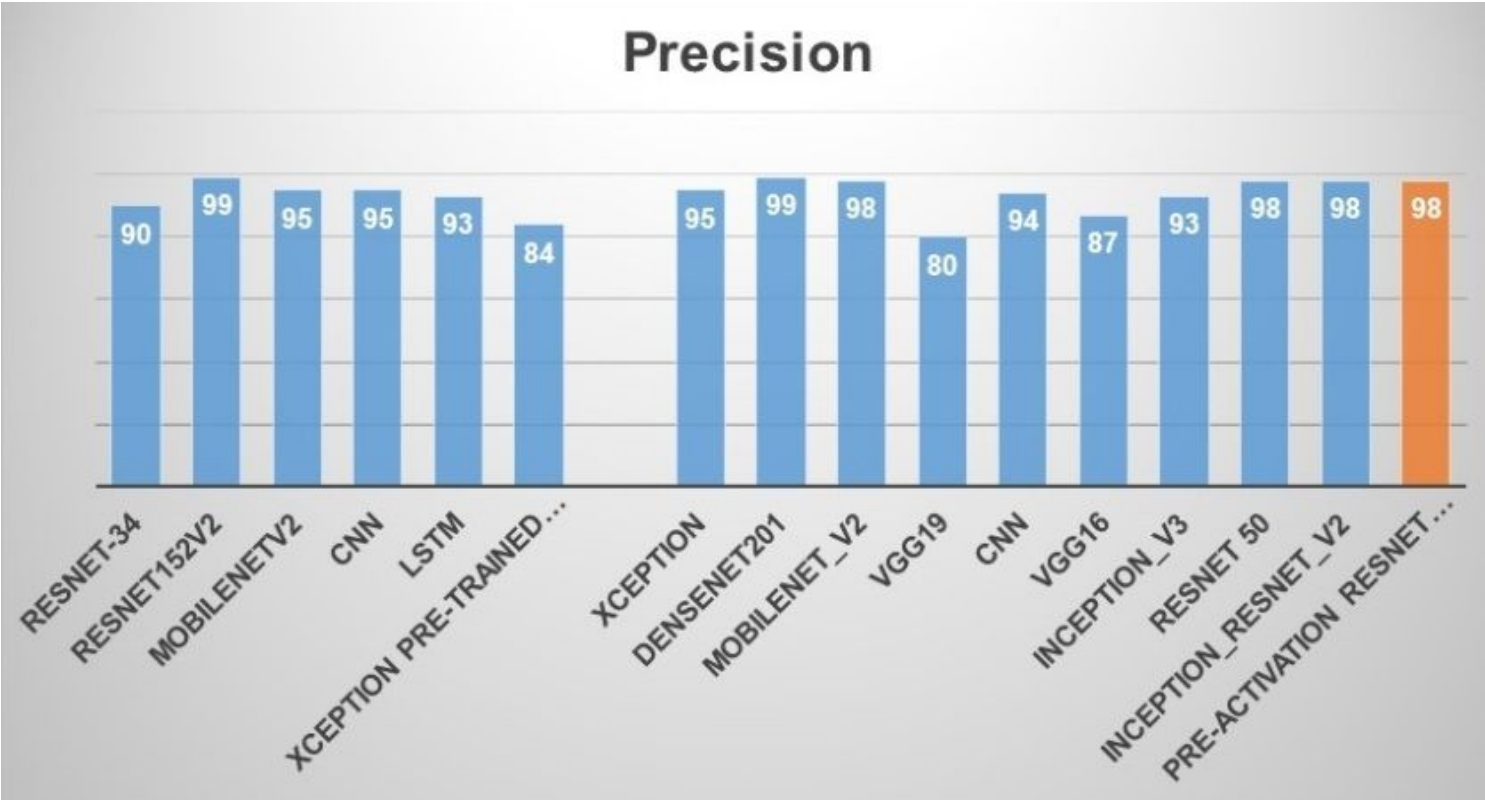


Figure 21

A comparison results with the Precision metric

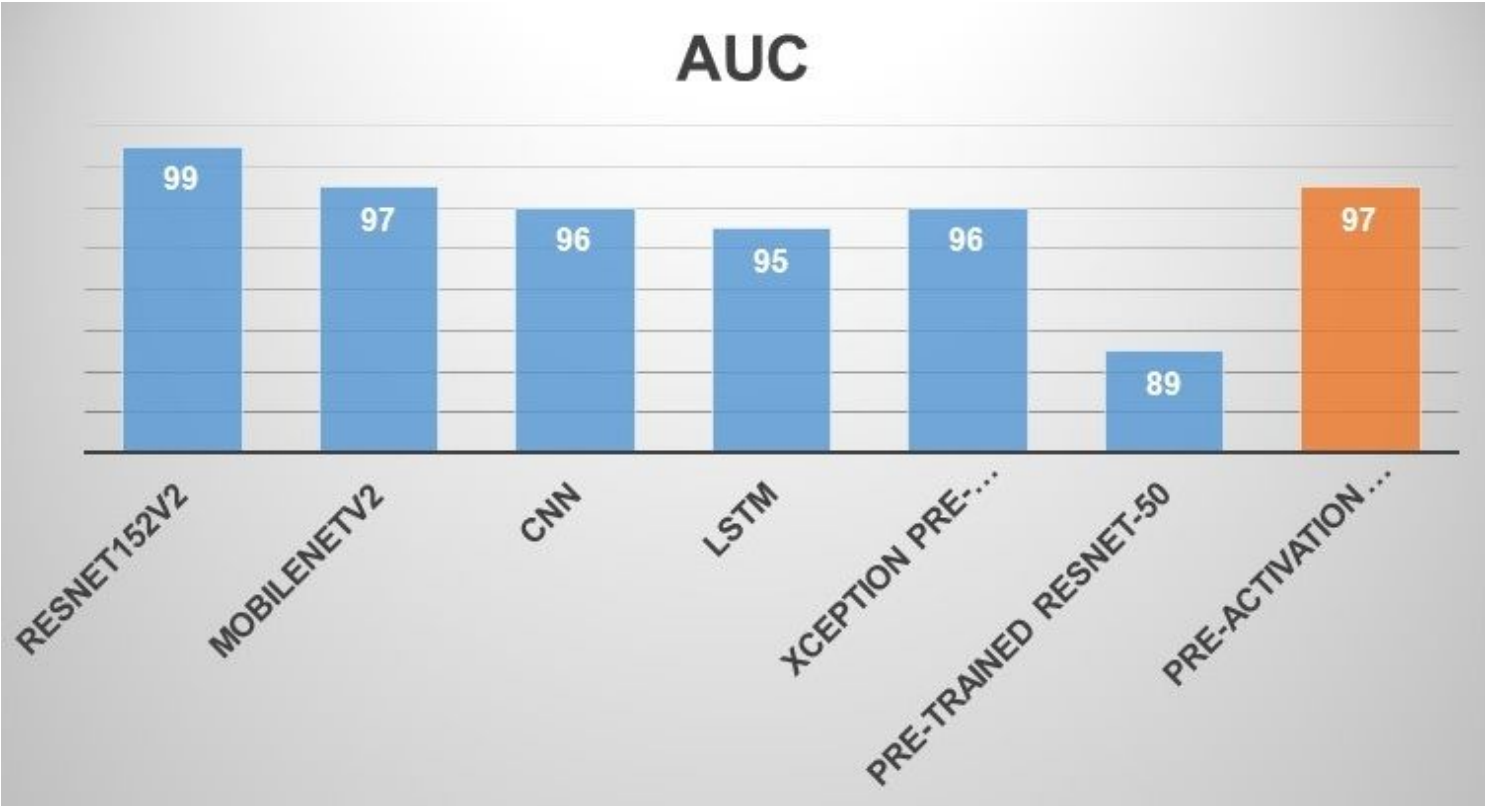


Figure 22

A comparison results with the AUC metric

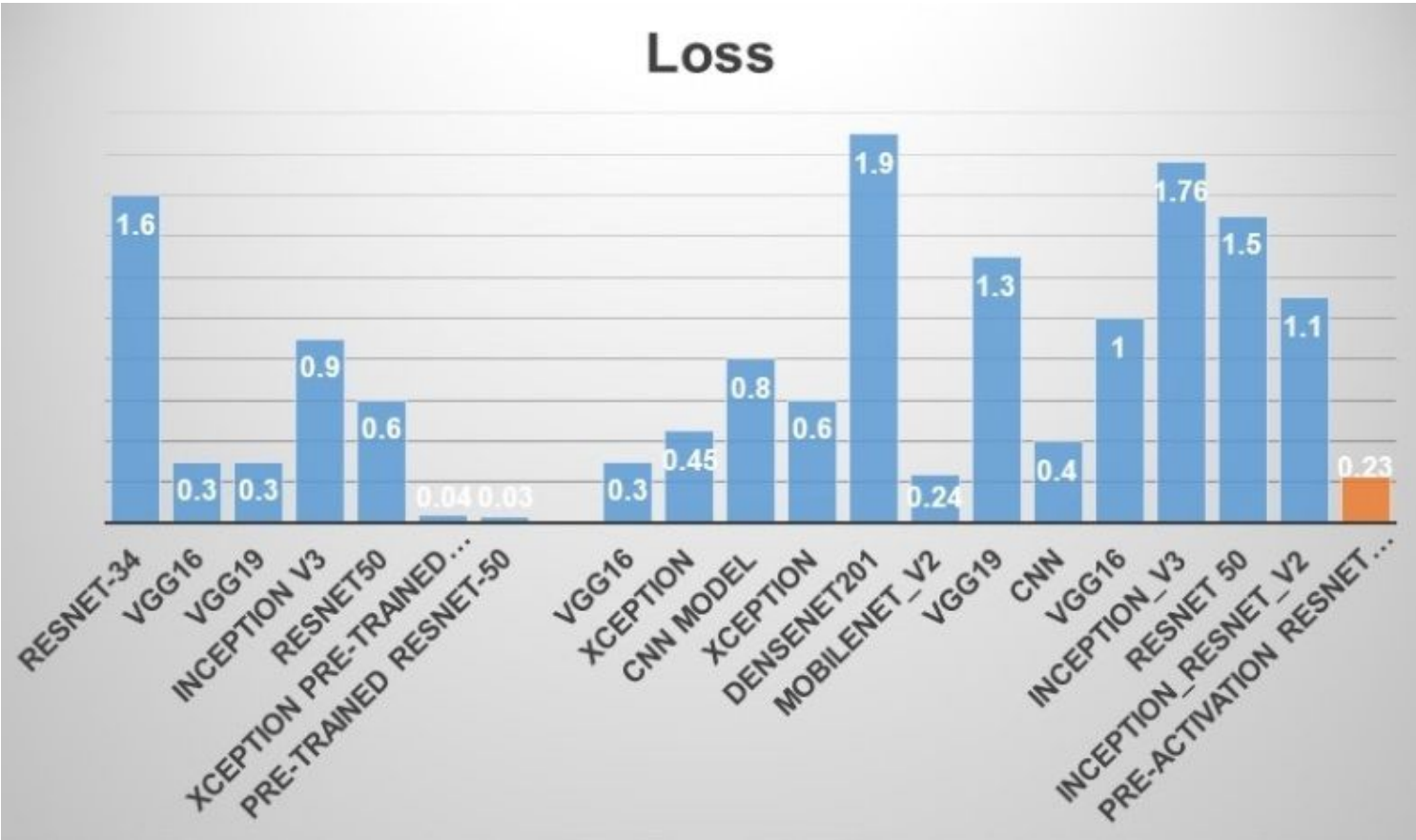


Figure 23

A comparison results with the loss metric